

Angular HandsOn 8: HTTP Client Integration and Interceptors

HANDS-ON 8 [Advanced]

HTTP Client ? API Integration, Observables & Interceptors

Topics Covered:

Setting Up HttpClientModule[Check] GET, POST, PUT, DELETE with HttpClient[Check]

Observables vs Promises[Check] RxJS Operators ? map, catchError, tap, [Check]

switchMap

HTTP Interceptors[Check] Error Handling & Retry Strategies[Check]

Real Angular applications communicate with backend APIs. In this hands-on, you will replace all hardcoded service data with live HTTP calls, handle errors gracefully, and add interceptors for cross-cutting concerns like auth tokens and logging.

SET

UP

Install JSON Server t- mock the backend: `npm install -g json-server`. Create `db.json` with `courses`, `students`, and `enrollments` arrays. Run: `json-server --watch db.json --port 3000`

Task 1: Replace Service Data with HttpClient Calls

Goal: Refactor CourseService and EnrollmentService t- use real HTTP calls.

Steps:

78. Import HttpClientModule in your app (or provideHttpClient() in app.config.ts for standalone Angular).

Inject HttpClient int- CourseService: `constructor(private http: HttpClient) {}`.

79. Replace the hardcoded courses array with an HTTP call: `getCourses(): Observable<Course[]> { return this.http.get<Course[]>('http://localhost:3000/courses'); }`. D- the same for `getCourseById`, returning `Observable<Course>`.

80. In CourseListComponent, update the `ngOnInit` t- subscribe:

```
this.courseService.getCourses().subscribe({ next: courses => this.courses = courses, error: err => this.errorMessage = err.message, complete: () => this.isLoading = false });
```

81. Add a POST method t- CourseService: `createCourse(course: Omit<Course, 'id'>):`

`Observable<Course> { return this.http.post<Course>('http://localhost:3000/courses', course); }`. Wire it t- the enrollment form's submit handler.

82. Add PUT and DELETE methods: `updateCourse` and `deleteCourse`. Test all four operations using Postman or the Angular app UI.

Hint:

- HttpClient methods return cold Observables ? they d- not execute until subscribed. Always unsubscribe or use the async pipe t- avoid memory leaks.

- Use the async pipe in templates (`{{ courses$ | async }}`) t- automatically subscribe and unsubscribe ? it is the cleanest pattern for displaying Observable data.

Expected Outcome: Course list loads from JSON Server. Adding a course via POST persists it t- db.json.

Angular HandsOn 8: HTTP Client Integration and Interceptors

DELETE removes it. All operations reflected in the UI.

Task 2: RxJS Operators and Error Handling

Goal: Apply RxJS operators to transform and handle HTTP responses robustly.

Steps:

83. Chain the map operator to transform the API response before it reaches the component:

`getCourses().pipe(map(courses => courses.filter(c => c.credits > 0)))`. Import map from 'rxjs/operators'.

84. Add error handling using `catchError`: `getCourses().pipe(catchError(err => { console.error(err); return throwError(() => new Error('Failed to load courses. Please try again.')). } )`. In the component, display the error message in the template.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book

85. Add a tap operator for side effects (logging): `getCourses().pipe(tap(courses => console.log('Courses loaded:', courses.length)), catchError(...))`. Explain in a comment why tap is preferred over side effects inside map.

86. Implement a retry strategy: `getCourses().pipe(retry(2), catchError(...))`. This retries failed HTTP requests up to 2 times before propagating the error.

87. Use `switchMap` to chain two HTTP calls: when a course is selected, load its enrolled students ? `switchMap(courseId => this.enrollmentService.getStudentsByCourse(courseId))`. Explain in a comment why `switchMap` cancels the previous inner Observable when a new courseId arrives.

Hint:

- `switchMap` is essential for type-ahead search and dependent HTTP calls ? it cancels the previous request if a new one arrives before the first completes, preventing out-of-order responses.

- `tap` is for side effects (logging, analytics) that should not alter the stream. Never modify data inside tap ? use map for transformations.

Expected Outcome: Courses load with error handling ? turning off JSON Server shows the error message.

retry attempts 2 retries before showing the error. `switchMap` cancels previous course student requests.

Task 3: HTTP Interceptors

Goal: Build interceptors to handle auth tokens and global error logging.

Steps:

88. Generate an interceptor: `ng generate interceptor interceptors/auth`. In the intercept method, clone the request and add an Authorization header: `req.clone({ setHeaders: { Authorization: 'Bearer mock-token-12345' } })`. Register it in `app.config.ts`: `provideHttpClient(withInterceptors([authInterceptor]))`.

89. Verify the interceptor works: open Chrome DevTools ? Network ? select any API call ? check the Request Headers for the Authorization header.

90. Generate a second interceptor: `ng generate interceptor interceptors/error-handler`. In it, use `catchError` to intercept HTTP errors globally: if status 401, navigate to login; if status 500, show a global error

Angular HandsOn 8: HTTP Client Integration and Interceptors

notification. Return throwError() to propagate after handling.

91. Generate a loading interceptor: ng generate interceptor interceptors/loading. Use a LoadingService (which you also create) with an isLoading\$ BehaviorSubject. Set it to true in intercept before next.handle(req), and false in the pipe's finalize operator. Bind a global loading spinner to loadingService.isLoading\$ using the async pipe.

Hint:

- Interceptors run in the order they are registered. The request goes through interceptors in registration order; the response travels back in reverse order.
- finalize in RxJS runs whether the Observable completes or errors ? it is the correct place to hide a loading spinner, equivalent to a try/catch/finally block.

Expected Outcome: DevTools shows Authorization header on all API requests. Simulating a 401 response navigates to home. A spinner appears on every HTTP call and disappears when it completes.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book